



# An Introduction to STL

---

# The C++ Standard Template Libraries

- ❖ In 1990, Alex Stepanov and Meng Lee of Hewlett Packard Laboratories extended C++ with a library of class and function templates which has come to be known as the STL.
- ❖ In 1994, STL was adopted as part of ANSI/ISO Standard C++.

# The C++ Standard Template Libraries

- ❖ STL had three basic components:
  - Containers

Generic class templates for storing collection of data.
  - Algorithms

Generic function templates for operating on containers.
  - Iterators

Generalized 'smart' pointers that facilitate use of containers.  
They provide an interface that is needed for STL algorithms to operate on STL containers.
- ❖ String abstraction was added during standardization.

# Why use STL?

- ❖ STL offers an assortment of containers
- ❖ STL publicizes the time and storage complexity of its containers
- ❖ STL containers grow and shrink in size automatically
- ❖ STL provides built-in algorithms for processing containers
- ❖ STL provides iterators that make the containers and algorithms flexible and efficient.
- ❖ STL is extensible which means that users can add new containers and new algorithms such that:
  - STL algorithms can process STL containers as well as user defined containers
  - User defined algorithms can process STL containers as well user defined containers

# Strings

- ❖ In C we used `char *` to represent a string.
- ❖ The C++ standard library provides a common implementation of a `string` class abstraction named `string`.

# Hello World - C

```
#include <stdio.h>

void main()
{
    // create string 'str' = "Hello world!"
    char *str = "Hello World!";

    printf("%s\n", str);
}
```

# Hello World - C++

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    // create string 'str' = "Hello world!"
    string str = "Hello World!";

    cout << str << endl;
    return 0;
}
```

# String

- ❖ To use the string type simply include its header file.

```
#include <string>
```

# Creating strings

```
string str = "some text";
```

or

```
string str("some text");
```

**other ways:**

```
string s1(7, 'a');
```

```
string s2 = s1;
```

# string length

The length of string is returned by its **size()** operation.

```
#include <string>
```

```
string str = "something";  
cout << "The size of "  
      << str  
      << "is " << str.size()  
      << "characters." << endl;
```

# The size method

`str.size() ???`

In C we had structs containing only data, In C++, we have :

```
class string
{
    ...
public:
    ...
    unsigned int size();
    ...
};
```

# String concatenation

concatenating one string to another is done by the '+' operator.

```
string str1 = "Here ";  
string str2 = "comes the sun";  
string concat_str = str1 + str2;
```

# String comparison

To check if two strings are equal use the '==' operator.

```
string str1 = "Here ";  
string str2 = "comes the sun";
```

```
if ( str1 == str2 )  
    /* do something */  
else  
    /* do something else */
```

# String assignment

To assign one string to another use the "=" operator.

```
string str1 = "Sgt. Pappers";  
string str2 = "lonely hearts club bend";  
str2 = str1;
```

Now : str2 equals "Sgt. Pappers"



# What more ?

---

- ❖ Containers
- ❖ Algorithms

# Containers

Data structures that hold **anything** (other objects).

- ❑ `list`: doubly linked list.
- ❑ `vector`: similar to a C array, but dynamic.
- ❑ `map`: set of ordered key/value pairs.
- ❑ `Set`: set of ordered keys.

# Algorithms

**generic** functions that handle common tasks such as searching, sorting, comparing, and editing:

- ☐ find
- ☐ merge
- ☐ reverse
- ☐ sort
- ☐ and more: count, random shuffle, remove, Nth-element, rotate.



# Vector

---

- ❖ Provides an alternative to the built in array.
- ❖ A vector is self grown.
- ❖ Use It instead of the built in array!

# Defining a new vector

Syntax: `vector<of what>`

For example :

`vector<int>` - vector of integers.

`vector<string>` - vector of strings.

`vector<int * >` - vector of pointers to integers.

`vector<Shape>` - vector of Shape objects. Shape is a user defined class.

# Using Vector

- ❖ `#include <vector>`
- ❖ Two ways to use the vector type:
  1. Array style.
  2. STL style

# Using a Vector - Array Style

We mimic the use of built-in array.

```
void simple_example()
{
    const int N = 10;
    vector<int> ivec(N);
    for (int i=0; i < 10; ++i)
        cin >> ivec[i];
}
```

# Using a vector - STL style

We define an empty vector

```
vector<string> svec;
```

we insert elements into the vector using the method `push_back`.

```
string word;
while ( cin >> word ) //the number of
                        words is unlimited.
{
    svec.push_back(word) ;
}
```

# Insertion

```
void push_back(const T& x);
```

Inserts an element with value *x* at the end of the controlled sequence.

```
svec.push_back(str);
```



# Size

```
unsigned int size();
```

Returns the length of the controlled sequence (how many items it contains).

```
unsigned int size = svec.size();
```



# Class Exercise 1

---

Write a program that read integers from the user, sorts them, and print the result.



# Solving the problem

---

- ❖ Easy way to read input.
- ❖ A “place” to store the input
- ❖ A way to sort the stored input.

# Using STL

```
int main()
{
    int input;
    vector<int> ivec;

    /* rest of code */
}
```

# STL - Input

---

```
while ( cin >> input )  
    ivec.push_back(input);
```

# STL - Sorting

---

```
sort(ivec.begin(), ivec.end());
```

Sort Prototype:

```
void sort(Iterator first, Iterator last);
```

# STL - Output

```
for ( int i = 0; i < ivec.size(); ++i )  
    cout << ivec[i] << " ";  
cout << endl;
```

Or (more recommended)

```
vector<int>::iterator it;  
for ( it = ivec.begin(); it != ivec.end(); ++it )  
    cout << *it << " ";  
cout << endl;
```

# STL - Include files

```
#include <iostream>    // I/O
#include <vector>       // container
#include <algorithm>    // sorting

//using namespace std;
```

# Putting it all together

```
int main() {
    int input;
    vector<int> ivec;

    // input
    while (cin >> input )
        ivec.push_back(input);

    // sorting
    sort(ivec.begin(), ivec.end());

    // output
    vector<int>::iterator it;
    for ( it = ivec.begin();
          it != ivec.end(); ++it ) {
        cout << *it << " ";
    }
    cout << endl;

    return 0;
}
```

# Operations on vector

- ❖ iterator begin();
- ❖ iterator end();
- ❖ bool empty();
- ❖ void push\_back(const T& x);
- ❖ iterator erase(iterator it);
- ❖ iterator erase(iterator first, iterator last);
- ❖ void clear();
- ❖ ....